Importance Resampling and Markov Chain Monte Carlo

Mark M. Fredrickson (mfredric@umich.edu)

Module 2: Generate

Computational Methods in Statistics and Data Science (DATASCI 406)

Densities Known up to a Constant

Known up to a constant

Often we can write a density as

$$f(x) = c\, f^*(x), \qquad c = \frac{1}{\int f^*(x)\, dx},$$

where f^* is easy to evaluate but the integral is not.

On day 3 the truncated normal had $f^*(x) = \phi(x)$ on $[a, b]$, and c needed $\Phi(b) - \Phi(a)$. In many dimensions the integral can be out of reach entirely.

Today: two samplers that use only f^*.

- **Importance resampling**, when we have an envelope g.
- **Markov chain Monte Carlo**, when we do not.

Importance Resampling

Resampling with importance weights

In module 1 we used the **importance weights** $f(Y_i)/g(Y_i)$ to estimate expectations. **Importance sampling resampling** (ISRS) uses them to pick samples instead.

Generate m samples Y_i from g. Pick a number J between 1 and m with probability

$$P(J = i \mid Y_i) = \frac{1}{m}\frac{f(Y_i)}{g(Y_i)}, \quad i = 1, \ldots, m$$

and set $X = Y_J$ (notice: all Y are random and J is random).

Note: $(1/m)f(Y_i)/g(Y_i)$ **might be larger than 1, we'll deal with this in a moment.**

Before the proof

1. Y_i has density $g(y)$.
2. $P(J = i) = \frac{1}{m}\frac{f(y)}{g(y)}$.
3. We will need to know the joint distribution of Y_i and J. They are independent so

$$g(y, j) = g(y) \cdot \frac{1}{m}\frac{f(y)}{g(y)} = \frac{1}{m}f(y)$$

Proof

The distribution of Y_J is the same as X. We need to show $P(Y_J \leq t) = P(X \leq t)$. How can the event $\{Y_J \leq t\}$ occur?

$$\begin{aligned}
\Pr(Y_J \leq t) &= P\left([J = 1, Y_1 \leq t] \text{ or } [J = 2, Y_2 \leq t] \text{ or } \ldots\right) && \text{(def. } Y_J\text{)} \\
&= \sum_{i=1}^{m} \Pr(Y_i \leq t, J = i) && \text{(disjoint events)} \\
&= \sum_{i=1}^{m} \frac{1}{m} \int_{-\infty}^{t} f(y)\, dy && \text{(fact 3)} \\
&= \int_{-\infty}^{t} f(y)\, dy = \Pr(X \leq t) && f \text{ is density of } X
\end{aligned}$$

Normalizing the weights

Problem: the weights $m^{-1} f(Y_i)/g(Y_i)$ may not be less than 1 and may not sum to 1.

We can normalize them as:

$$\omega_i = \frac{f(Y_i)/g(Y_i)}{\sum_{j=1}^{m} f(Y_j)/g(Y_j)}$$

The resulting Y_J will not have exactly the same distribution as X, but when m is large, the difference can be very small. This bias is the tradeoff for accepting all samples.

The constant cancels

If $f(x) = cf^*(x)$, the normalized weights do not change:

$$\omega_i = \frac{cf^*(Y_i)/g(Y_i)}{\sum_{j=1}^m cf^*(Y_j)/g(Y_j)} = \frac{f^*(Y_i)/g(Y_i)}{\sum_{j=1}^m f^*(Y_j)/g(Y_j)}$$

So ISRS draws from f using only f^*, and c never has to be computed.

Example: $Z \mid Z \geq 4.5$

The density of $Z \mid Z \geq 4.5$ is $\phi(z)/(1 - \Phi(4.5))$ for $z \geq 4.5$. We use only $f^*(z) = \phi(z)$, with shifted Exp(1) candidates $Y = 4.5 + E$.

```
k <- 10000
ys <- rexp(k) + 4.5
imp_weights <- dnorm(ys) / dexp(ys - 4.5)
omega <- imp_weights / sum(imp_weights)
xs <- sample(ys, replace = TRUE, prob = omega)
median(xs)
```

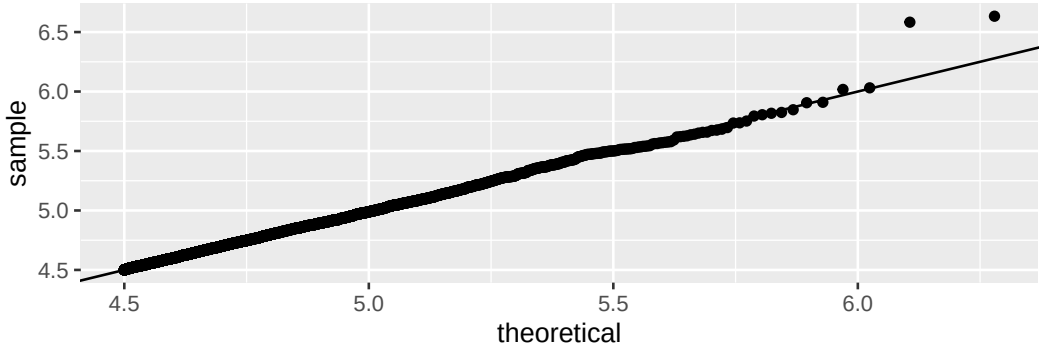
```
## [1] 4.646
```

Standard normal tails go to zero quickly: half of all Z larger than 4.5 are within 0.15 of 4.5.

Checking the draws

We compare against `qtnorm` from day 3, with $a = 4.5$ and $b = \infty$.

```
ggplot(data.frame(sample = xs), aes(sample = sample)) +
  geom_qq(distribution = qtnorm, dparams = list(a = 4.5, b = Inf)) +
  geom_abline(intercept = 0, slope = 1)
```



ISRS Summary

- Draw m candidates from an envelope g, weight each by $f^*(Y_i)/g(Y_i)$, and resample with probabilities proportional to the weights.
- Only f^* is needed: the constant cancels when the weights are normalized.
- No candidate is rejected. The cost is a small bias, which shrinks as m grows.
- g must be positive wherever f is, with tails at least as heavy. Otherwise a few candidates carry almost all the weight.
- When the quantile function is known, check the draws with a QQ plot.

Markov Chain Monte Carlo

Markov Chain Monte Carlo

We already know what **Monte Carlo** means.

The **Markov Chain** comes from the fact that we will draw samples from a **stochastic process**:

$$X(t) \,|\, X(t-1), X(t-2), \ldots, X(0) \sim X(t) \,|\, X(t-1)$$

(i.e., observation t only depends on observation $t-1$).

Such a stochastic process is called a **(discrete) Markov Chain**.

Goal: Markov chains whose averages converge to expectations under the target f, a law of large numbers:

$$\frac{1}{B} \sum_{b=1}^{B} h(X(b)) \overset{\text{a.s.}}{\to} E_f(h(X))$$

Achieving a SLLN

To achieve the desired result, we require Markov Chains that have the following properties:

- **Stationary distribution:** f is stationary if, when $X(t-1) \sim f$, $X(t) \sim f$.
- **Irreducible**: No matter the start of the chain $X(0)$, there is positive probability of visiting any region in the support of f.
- **Aperiodic**: The chain does not cycle through regions in a fixed order.

(The last two imply a condition called **ergodicity**, which you may see elsewhere.)

Luckily for us, we can use algorithms that already have these properties established!

A chain with a known stationary distribution

Day 4's autoregressive process, with $Z(t)$ independent $N(0,1)$:

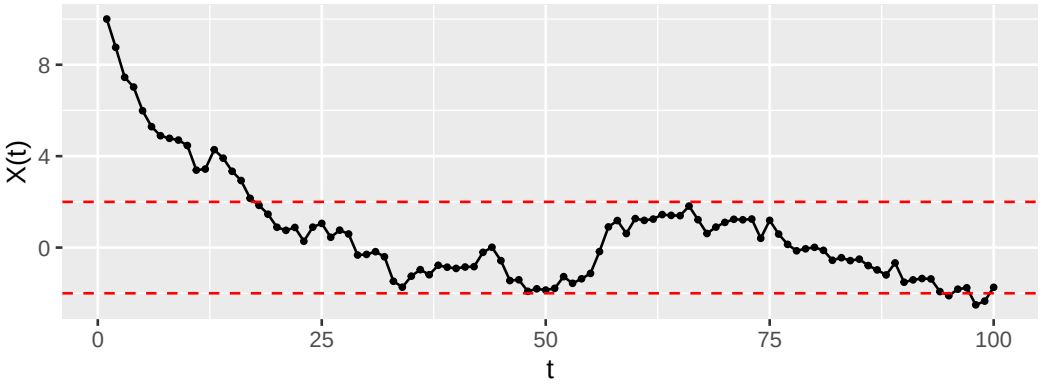
$$X(t) = \rho X(t-1) + \sqrt{1-\rho^2}\, Z(t)$$

If $X(t-1) \sim N(0,1)$, then $X(t) \sim N(0, \rho^2 + 1 - \rho^2) = N(0,1)$. So $N(0,1)$ is stationary.

Start far from it, at $X(0) = 10$:

```
rho <- 0.9
B <- 1000
chain <- numeric(B)
chain[1] <- 10
for (t in 2:B) {
  chain[t] <- rho * chain[t - 1] + sqrt(1 - rho^2) * rnorm(1)
}
```

The chain forgets where it started

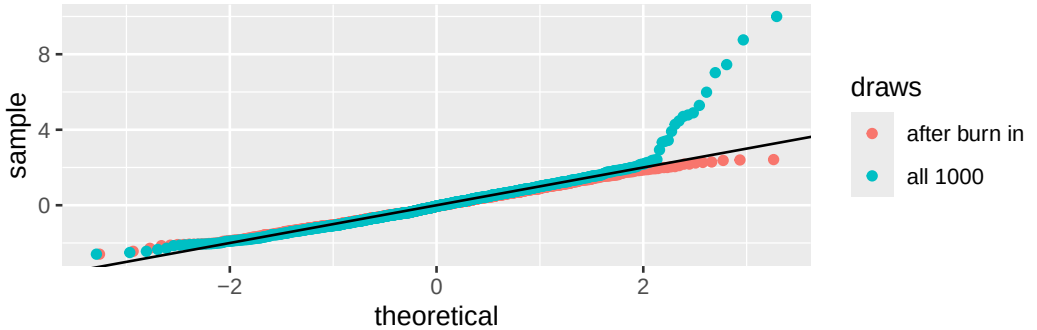


Each step keeps 90% of the distance from 0, so after about 40 steps the start no longer matters.

Burn in

The early draws come from the wrong distribution. Dropping them is called **burn in**.

```
tibble(sample = c(chain, chain[-(1:100)]),
       draws = rep(c("all 1000", "after burn in"), c(1000, 900))) %>%
  ggplot(aes(sample = sample, color = draws)) +
  geom_qq() + geom_abline(intercept = 0, slope = 1)
```



Metropolis-Hastings

Accept-reject Review

Recall the **accept-reject algorithm** for IID random variables:

- We wish to draw X from **target density** f but doing so is difficult.
- We pick **candidate density** g that we can draw from.
- We find a c such that $f(x)/cg(x) < 1$, for any x in the support of X
- Draw Y from g and U from $U(0,1)$
- Accept $X = Y$ if $U < f(Y)/cg(Y)$, reject and repeat otherwise.

Metropolis-Hastings: AR for MCMC

The **Metropolis-Hastings algorithm** applies the accept-reject concept to **Markov Chains**.

MH draws a candidate X^* from $g(\cdot \mid X(t-1))$ and either

1. Sets $X(t) = X^*$ if
$$U \leq \frac{f(X^*)}{f(X(t-1))} \frac{g(X(t-1) \mid X^*)}{g(X^* \mid X(t-1))}$$
2. Sets $X(t) = X(t-1)$, otherwise.

f appears only in a ratio, so f^* works in place of f.

Example: Rayleigh Density

Draw from the Rayleigh density:

$$f(x) = \frac{x}{\sigma^2} \exp\left\{\frac{-x^2}{2\sigma^2}\right\}, \quad x \geq 0, \sigma^2 > 0$$

To be clear: we want a sequence $X(0), X(1), \ldots$ that converges to f (we'll fix σ^2 to a constant).

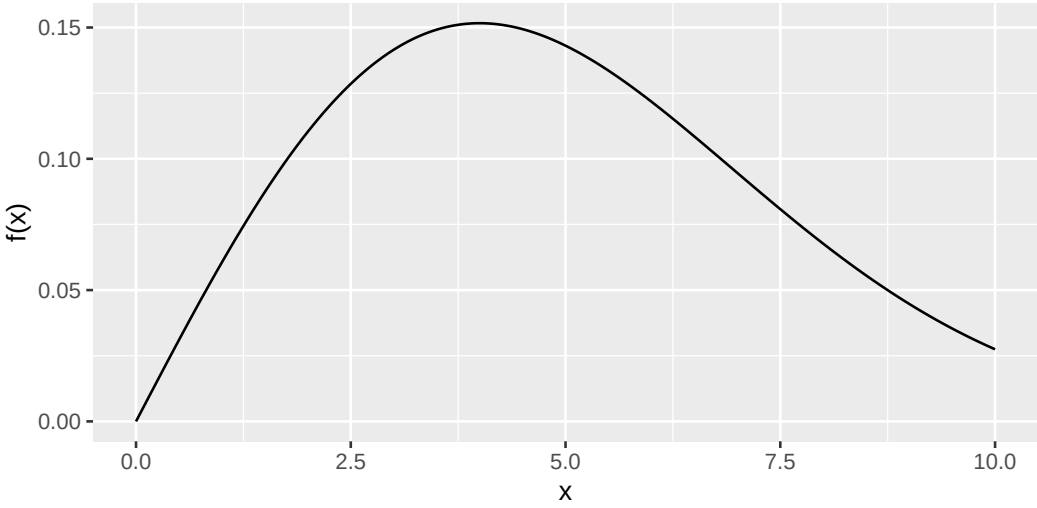
The main requirement we have for the **candidate distribution** is that it have the **same support** as the target.

It should also be **conditional on** $X(t-1)$.

We'll use a χ^2 distribution with $X(t-1)$ degrees of freedom.

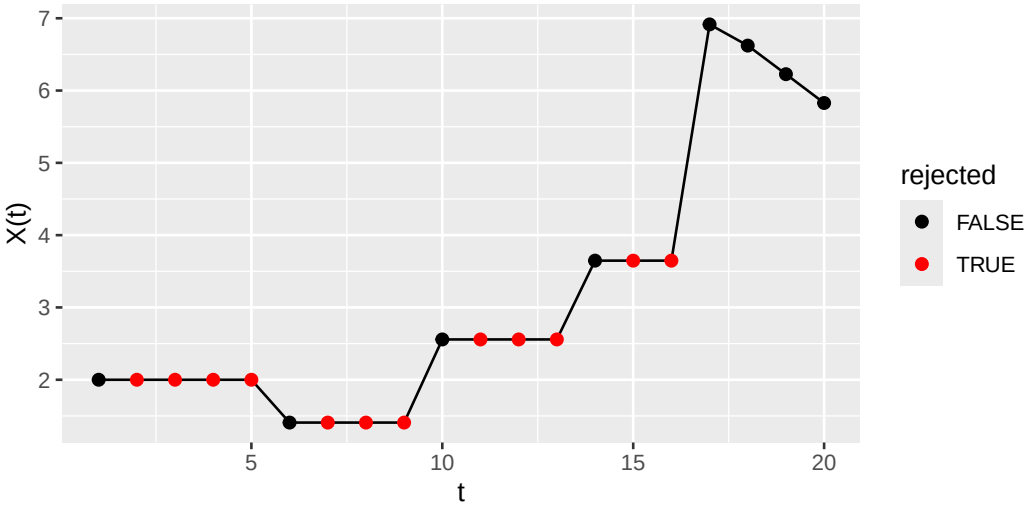
```r
# we'll fix sigma at 4
f <- function(x) {
  (x / 16) * exp(-x^2 / 32)
}
B <- 10000
xs <- numeric(B)
xs[1] <- 2 # arbitrary starting point

# we'll log rejects
rejected <- logical(B)
rejected[1] <- FALSE
```


0.15
0.10
0.05
0.00
f(x)
0.0
2.5
5.0
7.5
10.0
x

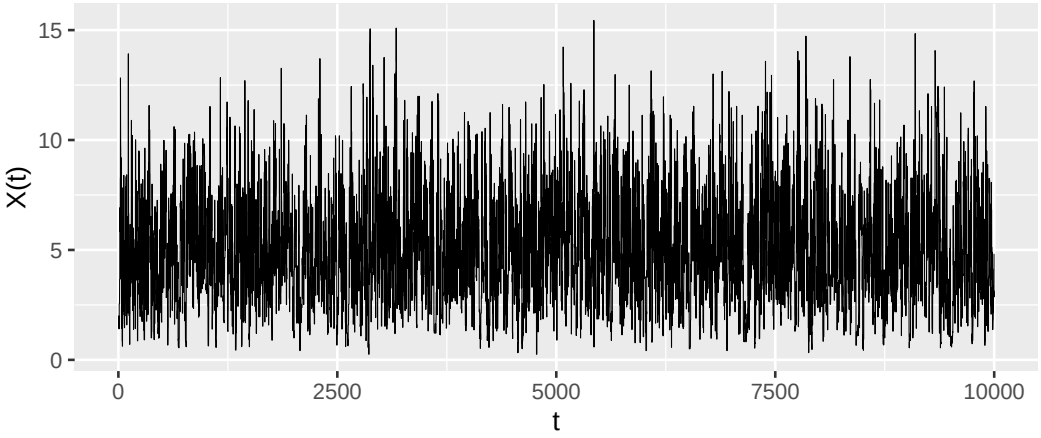
```r
# starting i = 2, apply MH
for (i in 2:B) {
  x <- xs[i - 1]
  xstar <- rchisq(1, df = x)
  ratio <- f(xstar) * dchisq(x, df = xstar) /
           (f(x) * dchisq(xstar, df = x))
  u <- runif(1)
  if (u <= ratio) {
    xs[i] <- xstar
    rejected[i] <- FALSE
  } else {
    xs[i] <- x
    rejected[i] <- TRUE
  }
}
```

Start of Chain



Trace plot

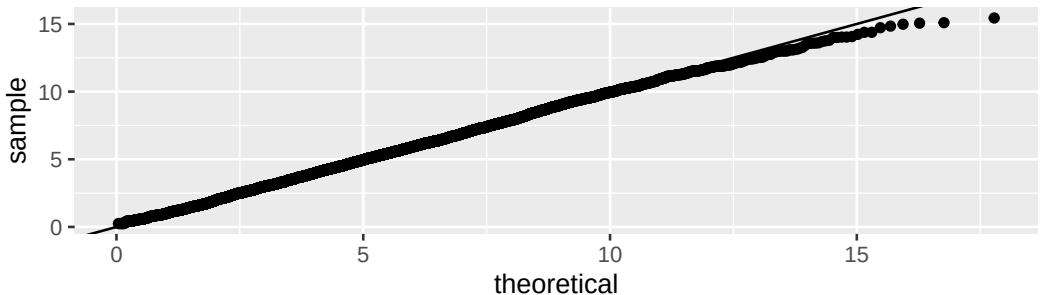
A **trace plot** shows the value of the chain against the iteration t.



Checking the chain

The Rayleigh CDF is $F(x) = 1 - e^{-x^2/2\sigma^2}$, so $Q(p) = \sigma\sqrt{-2\log(1-p)}$. Drop 200 draws as burn in:

```
q_rayleigh <- function(p) 4 * sqrt(-2 * log(1 - p))
ggplot(tibble(sample = xs[-(1:200)]), aes(sample = sample)) +
  geom_qq(distribution = q_rayleigh) +
  geom_abline(intercept = 0, slope = 1)
```



Gibbs Sampling

Gibbs sampling

It may be difficult to find effective multivariate candidates.

The **full conditional** of coordinate x_i is its distribution given all the others:

$$f_i(x_i) = f(x_i \mid x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_d)$$

As a function of x_i alone, $f_i(x_i) \propto f(x)$, so drop every factor of f that does not contain x_i.

Gibbs sampler: Step through the coordinates, drawing each x_i from f_i given the **current values of the rest**.

Special case of Metropolis-Hastings that

- draws x_i^* directly from f_i (which is always accepted)
- keeps $x_j(t) = x_j(t-1)$ for all $j \neq i$.

Gibbs for Multivariate Normal

If $(X_1, X_2, \ldots)$ are MVN, all **conditional distributions** are also normal.

Two dimensional case:

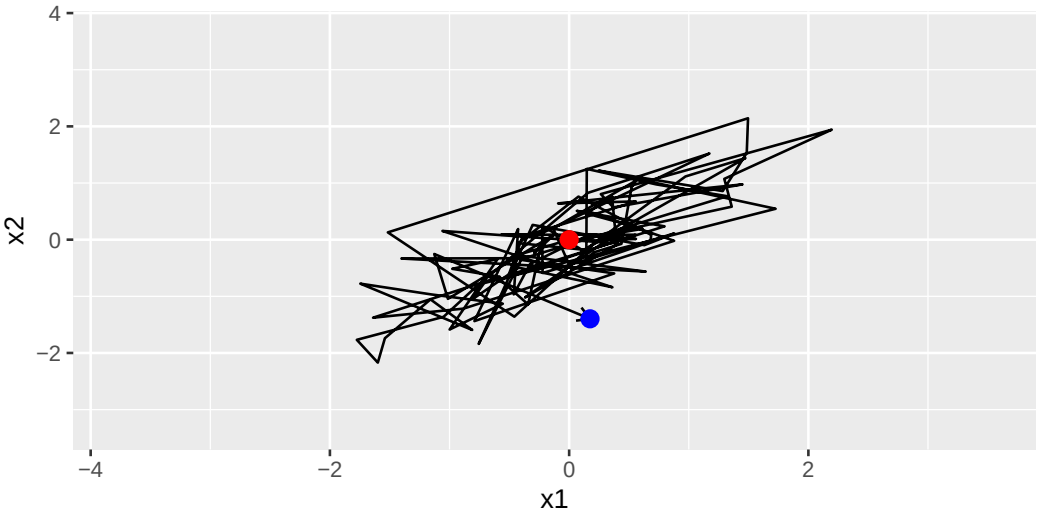
$$X_1 \,|\, X_2 = t \sim N\left(\mu_1 + \frac{\text{Cov}(X_1, X_2)}{\text{Var}(X_2)}(t - \mu_2), \text{Var}(X_1) - \frac{\text{Cov}(X_1, X_2)^2}{\text{Var}(X_2)}\right)$$

$$X_2 \,|\, X_1 = t \sim N\left(\mu_2 + \frac{\text{Cov}(X_2, X_1)}{\text{Var}(X_1)}(t - \mu_1), \text{Var}(X_2) - \frac{\text{Cov}(X_2, X_1)^2}{\text{Var}(X_1)}\right)$$

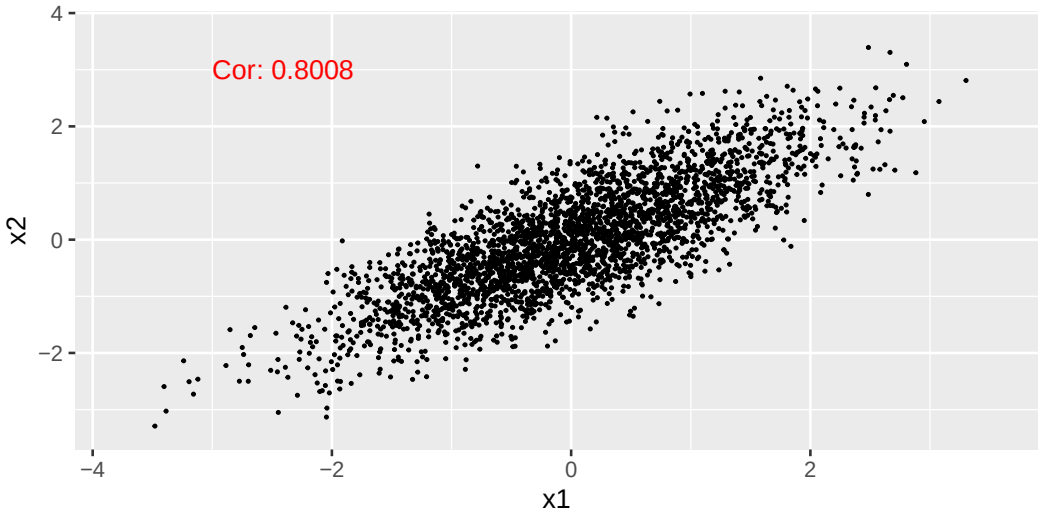
For example, take $\mu_1 = \mu_2 = 0$, $\text{Var}(X_1) = \text{Var}(X_2) = 1$, and $\text{Cov}(X_1, X_2) = 0.8$.

```r
B <- 5000
f_i <- function(t) {
  rnorm(1, mean = 0.8 * t, sd = sqrt(1 - 0.8^2))
}
chain_gibbs <- matrix(0, ncol = B, nrow = 2)
for (i in 2:B) {
    chain_gibbs[1, i] <- f_i(chain_gibbs[2, i - 1])
    chain_gibbs[2, i] <- f_i(chain_gibbs[1, i])
}
```

Gibbs: 1:100



Gibbs: 2000:5000



Remarks

- Must be able to draw from each full conditional directly.
- Gibbs needs no envelope and rejects nothing, but successive draws are dependent, so it needs burn in like any chain.

Bayesian Statistics

Bayes' rule

Treat a parameter θ as random, with a **prior** density $p(\theta)$. After observing data x with likelihood $f(x \mid \theta)$, Bayes' rule gives the **posterior**:

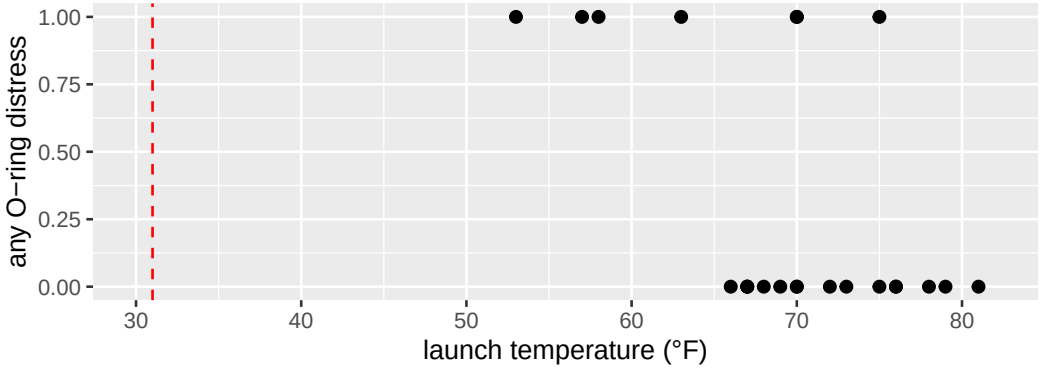
$$\pi(\theta \mid x) = \frac{f(x \mid \theta)\, p(\theta)}{\int f(x \mid \theta)\, p(\theta)\, d\theta} \propto f(x \mid \theta)\, p(\theta).$$

The denominator is a constant, so if we want to avoid computing it directly we can use:

- ISRS, using the prior as the envelope: the weight is the likelihood.
- Metropolis-Hastings, using $f(x \mid \theta)\, p(\theta)$ in place of f^*.
- Gibbs, when each parameter's full conditional is a known distribution.

The Challenger launch

The Space Shuttle Challenger broke up on 28 January 1986 after a booster field joint, sealed by O-rings, failed. The forecast launch temperature was 31°F. On the 23 earlier flights, no launch had been colder than 53°F.



What did the earlier flights say about the chance of O-ring distress at 31°F?

A logistic model

Let $y_i = 1$ if flight i had any O-ring distress, with

$$P(y_i = 1) = p_i, \qquad \log \frac{p_i}{1 - p_i} = \beta_0 + \beta_1(t_i - 70),$$

and independent $N(0, 5^2)$ priors on β_0 and β_1:

$$\pi(\beta_0, \beta_1 \mid y) \propto \prod_{i=1}^{23} p_i^{y_i}(1 - p_i)^{1-y_i}\ \phi(\beta_0/5)\,\phi(\beta_1/5).$$

We know no formula for the integral of the right side over (β_0, β_1), so we know this posterior only up to a constant. We want the distribution of

$$p(31) = \frac{1}{1 + e^{-(\beta_0 + \beta_1(31-70))}}.$$

Metropolis-Hastings in two dimensions

```
y <- as.numeric(oring$distress > 0)
x <- oring$temp - 70
log_post <- function(b) {
  sum(dbinom(y, 1, plogis(b[1] + b[2] * x), log = TRUE)) +
    sum(dnorm(b, 0, 5, log = TRUE))
}
B <- 20000
chain <- matrix(0, B, 2)
for (i in 2:B) {
  prop <- chain[i - 1, ] + rnorm(2, sd = c(0.6, 0.08))
  keep <- log(runif(1)) <= log_post(prop) - log_post(chain[i - 1, ])
  chain[i, ] <- if (keep) prop else chain[i - 1, ]
}
post <- chain[-(1:2000), ]
```

The chance of distress at 31°F

```
p31 <- plogis(post[, 1] + post[, 2] * (31 - 70))
quantile(p31, c(0.025, 0.5, 0.975))
```

```
##    2.5%     50%   97.5%
## 0.8798 0.9999 1.0000
```

```
mean(post[, 2] < 0)    # P(beta_1 < 0 | y): colder is worse
```

```
## [1] 0.9976
```

Every flight in the data was at least 22°F warmer than 31°F. The answer depends on the logistic model's form well outside the data, not only on the 23 flights.

Checking with a grid

With only two parameters, we can evaluate the posterior on a grid and normalize it by summing, which gives an independent answer:

```
grid <- expand.grid(b0 = seq(-6, 3, length.out = 300),
                    b1 = seq(-1, 0.3, length.out = 300))
lp <- apply(grid, 1, log_post)
w <- exp(lp - max(lp))
grid_p31 <- plogis(grid$b0 + grid$b1 * (31 - 70))
o <- order(grid_p31)
grid_p31[o][findInterval(c(0.025, 0.5, 0.975), cumsum(w[o]) / sum(w)) + 1]
```

```
## [1] 0.8992 0.9999 1.0000
```

A grid needs 300^d points in d dimensions, so this check stops working quickly as d grows.

Why not importance resampling?

Using the prior as the envelope, the weights are the likelihood:

```
m <- 100000
b0 <- rnorm(m, 0, 5); b1 <- rnorm(m, 0, 5)
ll <- sapply(1:m, function(k) {
  sum(dbinom(y, 1, plogis(b0[k] + b1[k] * x), log = TRUE))
})
w <- exp(ll - max(ll))
sum(w)^2 / sum(w^2)   # effective sample size
```

```
## [1] 539.6
```

The prior is much wider than the posterior, so a few hundred of the 100,000 candidates carry nearly all the weight. MCMC spends its draws where the posterior is.

Bayesian remarks

- Most posteriors, like this one, have no known closed form. With two parameters a grid can compute the constant numerically. With more, it cannot, and we rely on checks like trace plots.
- Both samplers work on the log scale, because a product of 23 probabilities can be too small to store accurately.
- Gibbs sampling needs full conditionals we can draw from. Choosing priors that are conjugate for each conditional is one common way to get them.
- **Hierarchical models**, in which the parameters of one level have their own prior, are a common use of Gibbs sampling.